

AVR LCD I2C Library (PCF8574)

1. Introdução

A biblioteca `lcd_i2c` é uma camada de abstração de alto nível para controle de displays LCD baseados no controlador HD44780 utilizando expansores de I/O PCF8574 via barramento I²C (TWI).

O diferencial arquitetônico fundamental é a injeção de dependência. Ao contrário de bibliotecas comuns, a `lcd_i2c` não possui código de hardware fixo. Ela utiliza ponteiros de função para que o usuário forneça as rotinas de escrita I²C e Delay. Com isso, pretende-se que a mesma biblioteca funcione em microcontroladores AVR, STM32, ESP32 e outros sem alteração no código-fonte da biblioteca.

2. Estrutura de Arquivos

A biblioteca é composta por quatro arquivos fundamentais:

- **lcd_i2c_types.h**: Definições de tipos, enums de status e a struct `lcd_t` (o "objeto" LCD).
- **lcd_i2c_config.h**: Definições de hardware (mapeamento de pinos do PCF8574 e comandos do HD44780).
- **lcd_i2c_internal.h**: Protótipos de funções de baixo nível (manipulação de nibbles e pulso de Enable).
- **lcd_i2c.h**: API Pública (Funções que o desenvolvedor chamará no `main.c`).
- **lcd_i2c.c**: Implementação da lógica de controle.

3. Requisitos de Implementação

Para utilizar a biblioteca, o desenvolvedor deve fornecer duas funções de suporte no seu projeto principal:

3.1. O Wrapper I²C (i2c_write)

A biblioteca precisa enviar bytes para o expensor de I/O. Como ela não sabe qual driver I²C ou TWI o desenvolvedor usa, ela espera uma função com esta assinatura:

```
int8_t minha_funcao_escrita(uint8_t endereco, uint8_t dado);
```

3.2. O Wrapper de Delay (delay_ms)

Macros como por exemplo: `_delay_ms()` do AVR, não aceitam variáveis. Por isso, a biblioteca espera uma função que processe atrasos dinâmicos:

```
void meu_delay_customizado(uint16_t ms) {  
    while(ms--) _delay_ms(1);  
}
```

4. API Reference (Funções do Usuário)

lcd_init

```
lcd_status_t lcd_init(lcd_t *handle);
```

- **O que faz:** Realiza a sequência complexa de "Reset por Instrução" para colocar o LCD em modo de 4 bits. Configura o display para 2 linhas e limpa a tela.
- **Importante:** Deve ser chamada após a inicialização do barramento TWI físico.

lcd_write_string

```
void lcd_write_string(lcd_t *handle, const char *str);
```

- **O que faz:** Recebe uma string (vetor de caracteres) e a envia caractere por caractere para o display.

lcd_set_cursor

```
void lcd_set_cursor(lcd_t *handle, uint8_t col, uint8_t row);
```

- **O que faz:** Move o cursor para a posição desejada.
- **Parâmetros:** col (0-15) e row (0 para linha superior, 1 para inferior).

lcd_backlight

```
void lcd_backlight(lcd_t *handle, lcd_backlight_t state);
```

- **O que faz:** Liga (LCD_BACKLIGHT_ON) ou desliga (LCD_BACKLIGHT_OFF) a luz de fundo.
- **Por que usar:** Útil para economia de energia ou alertas visuais (piscar o backlight).

lcd_create_custom_char

```
void lcd_create_custom_char(lcd_t *handle, uint8_t location, uint8_t charmap[]);
```

- **O que faz:** Grava um desenho personalizado na memória CGRAM do LCD.
- **Parâmetros:** location (índice de 0 a 7) e charmap (array de 8 bytes com o desenho).

lcd_display_control

```
void lcd_display_control(lcd_t *handle, bool cursor_on, bool blink_on);
```

- **O que faz:** Ativa/desativa visualmente o cursor (sublinhado) e o efeito de piscar (bloco preenchido).

5. Exemplo Prático de Aplicação

Abaixo, o fluxo padrão de configuração no main.c:

```
#include "twi.h"
```

```
#include "lcd_i2c.h"
```

```
// 1. Criamos as pontes de hardware
```

```
int8_t bridge_i2c(uint8_t addr, uint8_t data) {  
    return (twi_write_byte(addr, data) == TWI_OK) ? 0 : -1;  
}
```

```
void bridge_delay(uint16_t ms) {  
    while(ms--) _delay_ms(1);  
}
```

```

int main() {
    // 2. Criamos e configuramos o objeto (Handle)
    lcd_t lcd1;
    lcd1.address = 0x27;
    lcd1.columns = 16;
    lcd1.rows = 2;
    lcd1.backlight = LCD_BACKLIGHT_ON;
    lcd1.i2c_write = bridge_i2c; // Injeção da função I2C
    lcd1.delay_ms = bridge_delay; // Injeção da função Delay

    // 3. Inicializamos
    if (lcd_init(&lcd1) == LCD_OK) {
        lcd_write_string(&lcd1, "Status: OK!");
    }

    while(1);
}

```

6. Considerações de Hardware (Módulo I²C)

A biblioteca assume o mapeamento padrão de mercado para o expensor **PCF8574**:

- **P0:** RS (Register Select)
- **P1:** RW (Read/Write)
- **P2:** EN (Enable)
- **P3:** Backlight Control
- **P4-P7:** D4-D7 (Dados do LCD)

Caso seu hardware seja diferente, as alterações devem ser feitas exclusivamente no arquivo `lcd_i2c_config.h`.

7. Conclusão

Essa biblioteca foi projetada para ser:

- **Fácil de usar** (API intuitiva)
- **Flexível** (Pode ser usada em diferentes famílias de microcontroladores)

Boa programação e bons projetos!

Tiago Henrique dos Santos

Contatos:

Canal do YouTube: <https://www.youtube.com/channel/UCiIEwgsH0HEj3P2aA1-Sozw>

Instagram: <https://www.instagram.com/cienciaeletrica/>

E-mail: contato.cienciaeletrica@gmail.com